KCC CODE CLUB · GAME MAKERS

Concept Map

What is taught when

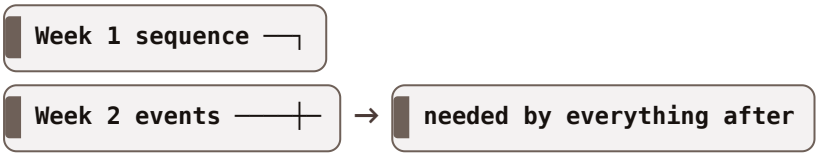
What is taught when, and what each week depends on. Use this to check that an activity you're inventing doesn't need a block the class hasn't met yet — the most common way a well-meaning volunteer accidentally derails a session.

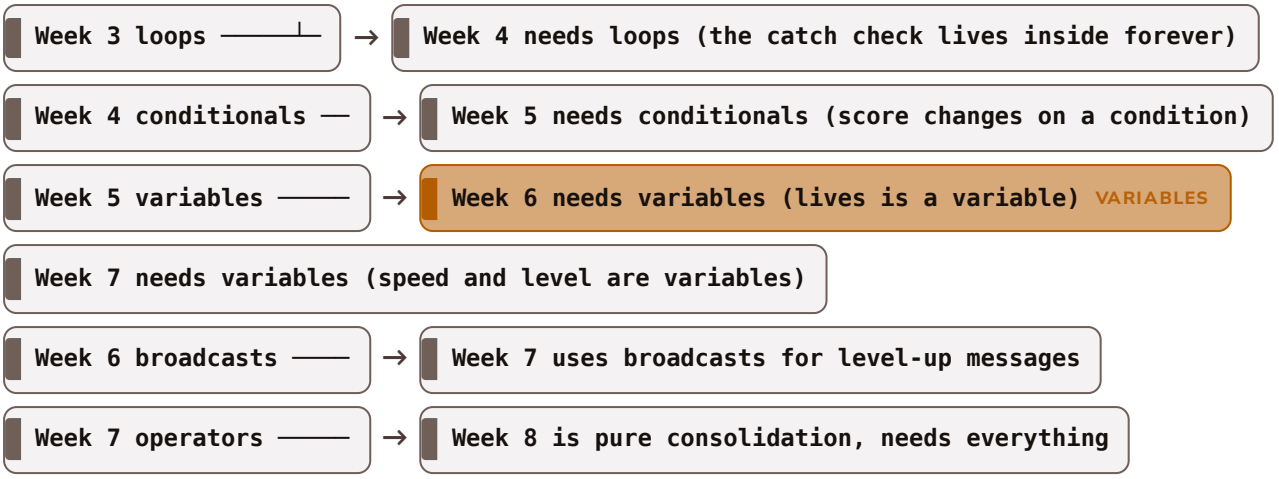
Concepts by week

Week	Computer science idea	Scratch blocks introduced
1	Sequence — order matters. A program is instructions carried out in order.	`when green flag clicked`, `move`, `turn`, `go to x y`, `say`, `say for n secs`, `think`, `switch costume`, `change size by`, `next costume`
2	Events — code triggered by something happening. Coordinates as a way to describe position.	`when key pressed`, `when this sprite clicked`, `change x by`, `change y by`, `set x to`, `set y to`, `glide n secs to x y`, `point in direction`
3	Iteration — repeating without repeating yourself. Randomness.	`repeat n`, `forever`, `wait n secs`, `pick random a to b`, `repeat until`
4	Selection — the computer making a decision. Booleans as yes/no questions. Sensing the world.	`if... then`, `if... then... else`, `touching [sprite]?`, `touching [edge]?`, `key [x] pressed?`, `start sound`, `play sound until done`
5	Variables — a named box that holds a value and can change.	`set [var] to`, `change [var] by`, `show variable`, `hide variable`, variable reporter blocks
6	Message passing — separate parts of a program coordinating. Program state.	`broadcast`, `broadcast and wait`, `when I receive`, `switch backdrop to`, `stop all`, `stop this script`, `hide`, `show`
7	Operators and abstraction — building expressions, and naming a group of steps as one new instruction.	`+ − × /`, `< = >`, `and`, `or`, `not`, `join`, `ask and wait`, `answer`, My Blocks (`define`)
8	Debugging and user experience. Sharing and remixing.	No new blocks. Consolidation, testing, and publishing

Dependency chain

Nothing in week *n* requires anything from week *n+1*. The chain is strictly forward, which is what makes the skeleton catch-up files work.





Two deliberate exceptions to the chain

Both are fine, but know about them so you're not caught out mid-session.

Comparison operators appear before week seven. `=`, `>` and `<` are formally taught in week seven, but they turn up earlier because you can't build a score without them: the week-five click counter bonus uses `if <(clicks) = 10>`, the week-five high-score homework uses `>`, and the week-six lives check uses `< 1`. This is intentional. Children read `>` and `<` intuitively long before they can nest operator blocks inside each other, which is the actual week-seven skill. Introduce them casually when they first appear and don't make a lesson of it.

`if on edge, bounce` is a Motion block, not a conditional. It shows up in the week-three dancing sprite bonus, a week before `if` is taught. It reads like a conditional and it isn't — it's a single motion block that happens to have the word "if" in its name. If a child asks, that's a good answer, and it plants the idea nicely for week four.

Vocabulary to use consistently

Volunteers drifting between synonyms confuses children more than the concepts do. Agree on these words and stick to them all term.

Use this	Not this
sprite	character, object, thing
script	code, program, stack (a *script* is one connected stack of blocks)
stage	screen, background (the *backdrop* is the picture; the *stage* is the area)
block	command, instruction, piece
variable	box, container, memory
broadcast	message, signal, event (reserve *event* for the hat blocks)
bug	error, mistake, problem
debug	fix, solve

"Bug" and "debug" are worth being deliberate about. Framing a broken program as *having a bug to find* rather than *being wrong* changes how children respond to failure, and that shift is arguably the most valuable thing they take away from the whole course.

What is deliberately NOT covered

Say so out loud in week eight, so the ambitious ones know where to go next.

- **Clones** — genuinely useful for this game, but the mental model (a sprite that copies itself) is a step too far for eight weeks. It appears once as a week-seven bonus challenge for anyone racing ahead.
- **Lists** — arrays are the natural next topic and there simply isn't room.
- **Custom block arguments** — My Blocks are introduced without inputs. Adding parameters is a bonus challenge.
- **The pen and music extensions** — mentioned in bonus challenges, never taught. They're a great rabbit hole for a curious kid to fall into at home.
- **Anything text-based.** If a student is hungry for text code by week six, point them at the Raspberry Pi Foundation's "Try coding with text" activity and, next term, the Python track.